

Answer Key

Exercise 1: b) 6

Exercise 2: b) 8

Exercise 3: See solution below

Exercise 4: See solution below

Exercise 5: See solution below

Exercise 6: See solution below

Exercise 7: See solution below

Exercise 8: See solution below

Exercise 9: d) ZYX

Exercise 10: b) CarBike

Exercise 11: See solution below

Exercise 12: See solution below

Exercise 13: See solution below

Exercise 14: See solution below

Exercise 15: b) Using `try-with-resources` statement

Exercise 16: d) The code should handle `IOException` exceptions

Solutions for Code Exercises

Exercise 3 - Solution

```
static int power(int x, int n) {  
    if (n == 0) {  
        return 1;  
    } else {  
        return x * power(x, n - 1);  
    }  
}
```

Exercise 4 - Solution

```
ArrayList<String> allSubstrings(String s) {  
    ArrayList<String> result = new ArrayList<String>();  
    if (s.length() == 0) {  
        result.add("");  
    } else {  
        // Get all substrings without first character  
        ArrayList<String> rest = allSubstrings(s.substring(1));  
        // Add all substrings from rest (not including first char)  
        result.addAll(rest);  
        // Add all substrings with first char  
        for (String str : rest) {  
            result.add(s.charAt(0) + str);  
        }  
    }  
}
```

```
    }  
    return result;  
}
```

Exercise 5 - Solution

```
static int findLast(Node p) {  
    // Find the first node  
    while (p.prev != null) {  
        p = p.prev;  
    }  
    // Traverse to last node  
    while (p.next != null) {  
        p = p.next;  
    }  
    return p.element;  
}
```

Exercise 6 - Solution

```
void addAtIndex(int i, int x) {  
    Node newNode = new Node();  
    newNode.element = x;  
  
    if (i == 0) {  
        newNode.next = this.first;  
        this.first = newNode;  
        return;  
    }  
  
    Node current = this.first;  
    for (int pos = 0; pos < i - 1; pos++) {  
        current = current.next;  
    }  
    newNode.next = current.next;  
    current.next = newNode;  
}
```

Exercise 7 - Solution

```
static <T> int treeHeight(TreeNode<T> root) {  
    if (root == null) {  
        return 0;  
    } else {  
        return 1 + Math.max(treeHeight(root.leftChild),  
            treeHeight(root.rightChild));  
    }  
}
```

```
}  
}
```

Exercise 8 - Solution

```
static <T> ArrayList<T> collectLeaves(TreeNode<T> root) {  
    ArrayList<T> result = new ArrayList<T>();  
    if (root == null) {  
        return result;  
    }  
    if (root.leftChild == null && root.rightChild == null) {  
        result.add(root.key);  
    } else {  
        result.addAll(collectLeaves(root.leftChild));  
        result.addAll(collectLeaves(root.rightChild));  
    }  
    return result;  
}
```

Exercise 11 - Solution

```
class LimitedCounter extends Counter {  
    private int maxValue;  
  
    LimitedCounter(int maxValue) {  
        super();  
        this.maxValue = maxValue;  
    }  
  
    @Override  
    void increment() {  
        if (this.value < this.maxValue) {  
            super.increment();  
        }  
    }  
}
```

Exercise 12 - Solution

```
class EvenPredicate implements Predicate<Integer> {  
    @Override  
    public boolean test(Integer element) {  
        return element % 2 == 0;  
    }  
}
```

Exercise 13 - Solution

```
static <T> boolean allMatch(Predicate<T> predicate, ArrayList<T> list) {  
    for (T element : list) {  
        if (!predicate.test(element)) {  
            return false;  
        }  
    }  
    return true;  
}
```

Exercise 14 - Solution

```
class FibonacciIterator implements Iterator<Integer> {  
    private int prev;  
    private int current;  
  
    FibonacciIterator() {  
        this.prev = 0;  
        this.current = 1;  
    }  
  
    @Override  
    public boolean hasNext() {  
        return true; // Infinite stream  
    }  
  
    @Override  
    public Integer next() {  
        int result = this.prev;  
        int next = this.prev + this.current;  
        this.prev = this.current;  
        this.current = next;  
        return result;  
    }  
}
```